Usn No: CS23O82

Mrunal Samrutwar

Aim: To design and implement an Artificial Neural Network for classification problem and evaluate its learning performance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,classification_report
```

```
from google.colab import drive
drive.mount("/content/drive")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
path ="/content/drive/MyDrive/ML datasets/churn.csv"
df=pd.read_csv(path)
df.head(5)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMemb
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	

Next steps: Generate code with df New interactive sheet

```
df.shape
```

(10000, 14)

```
df.columns
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

```
df.isnull().any().any()
```

np.False_

```
X = df.drop(columns=['Exited'])
y = df['Exited']
```

```
X.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMemb
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	

Next steps: Generate code with X New interactive sheet

```
X = pd.get_dummies(X,dtype = int)
```

```
X.head()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	...	Su
0	1	15634602	619	42	2	0.00	1	1	1	101348.88	...	
1	2	15647311	608	41	1	83807.86	1	0	1	112542.58	...	
2	3	15619304	502	42	8	159660.80	3	1	0	113931.57	...	
3	4	15701354	699	39	1	0.00	2	0	0	93826.63	...	
4	5	15737888	850	43	2	125510.82	1	1	1	79084.10	...	

5 rows × 2947 columns

```
X = df.iloc[:,3:-1]
X = pd.get_dummies(X, dtype=int)
X
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geography_France	Geography
0	619	42	2	0.00	1	1	1	101348.88	1	
1	608	41	1	83807.86	1	0	1	112542.58	0	
2	502	42	8	159660.80	3	1	0	113931.57	1	
3	699	39	1	0.00	2	0	0	93826.63	1	
4	850	43	2	125510.82	1	1	1	79084.10	0	
...	...	...	...	...	...	...	...	...	...	
9995	771	39	5	0.00	2	1	0	96270.64	1	
9996	516	35	10	57369.61	1	1	1	101699.77	1	
9997	709	36	7	0.00	1	0	1	42085.58	1	
9998	772	42	3	75075.31	2	1	0	92888.52	0	
9999	792	28	4	130142.79	1	1	0	38190.78	1	

10000 rows × 13 columns

Next steps: Generate code with X New interactive sheet

```
y=df.iloc[:,-1]
print(y)
```

```
0      1
1      0
2      1
3      0
4      0
      ..
9995   0
9996   0
9997   1
9998   1
9999   0
Name: Exited, Length: 10000, dtype: int64
```

```
X.shape
```

```
(10000, 13)
```

```
X.columns
```

```
Index(['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Geography_France',
       'Geography_Germany', 'Geography_Spain', 'Gender_Female', 'Gender_Male'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,train_size=0.7,test_size=0.3,random_state=42)
X_train.shape,X_test.shape
```

```
((7000, 13), (3000, 13))
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
X_train
```

```
array([[-0.34459497, -0.65674999, -0.34217046, ...,  1.73073215,
         1.09823226, -1.09823226],
       [-0.09518109, -0.46637979,  0.69816249, ..., -0.57779016,
         1.09823226, -1.09823226],
       [-0.94734518, -0.56156489,  0.35138484, ..., -0.57779016,
        -0.91055421,  0.91055421],
       ...,
       [ 0.86090545, -0.08563939, -1.38250341, ..., -0.57779016,
         1.09823226, -1.09823226],
       [ 0.15423279,  0.39028611,  1.04494014, ..., -0.57779016,
        -0.91055421,  0.91055421],
       [ 0.46600014,  1.1517669 , -1.38250341, ..., -0.57779016,
        -0.91055421,  0.91055421]])
```

```
y_train
```

	Exited
9069	1
2603	0
7738	0
1579	0
5058	0
...	...
5734	0
5191	0
5390	1
860	1
7270	0

7000 rows × 1 columns

dtype: int64

```
import tensorflow as tf
```

```
#initialising ANN
ann = tf.keras.models.Sequential()
```

```
#adding first hidden layer
ann.add(tf.keras.layers.Dense(units=6,activation="relu"))
```

```
#adding second hidden layer
ann.add(tf.keras.layers.Dense(units=6,activation="relu"))
```

```
#Adding output layer
ann.add(tf.keras.layers.Dense(units=1,activation="sigmoid"))
```

```
#compiling the ann
ann.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
```

```
#fitting  the ann to the training set
ann.fit(X_train,y_train,batch_size=32,epochs=100)
```

```
Epoch 68/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8654 - loss: 0.3288
Epoch 69/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8620 - loss: 0.3326
Epoch 70/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8700 - loss: 0.3318
Epoch 71/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8579 - loss: 0.3426
Epoch 72/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8554 - loss: 0.3542
Epoch 73/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8697 - loss: 0.3213
Epoch 74/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8630 - loss: 0.3367
Epoch 75/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8631 - loss: 0.3329
Epoch 76/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8651 - loss: 0.3285
Epoch 77/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8610 - loss: 0.3403
Epoch 78/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8629 - loss: 0.3340
Epoch 79/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8637 - loss: 0.3310
Epoch 80/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8584 - loss: 0.3361
Epoch 81/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8654 - loss: 0.3300
Epoch 82/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8621 - loss: 0.3287
Epoch 83/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8651 - loss: 0.3329
Epoch 84/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8602 - loss: 0.3290
Epoch 85/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8665 - loss: 0.3360
Epoch 86/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8616 - loss: 0.3335
Epoch 87/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8669 - loss: 0.3263
Epoch 88/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8630 - loss: 0.3294
Epoch 89/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8634 - loss: 0.3286
Epoch 90/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8605 - loss: 0.3408
Epoch 91/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8658 - loss: 0.3344
Epoch 92/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8642 - loss: 0.3360
Epoch 93/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8629 - loss: 0.3342
Epoch 94/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8666 - loss: 0.3308
Epoch 95/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8670 - loss: 0.3196
Epoch 96/100
219/219 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8569 - loss: 0.3401
```

```
y_pred = ann.predict(X_test)
y_pred = (y_pred > 0.5)
y_pred
```

```
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

```
array([[False],
       [False],
       [False],
       ...,
       [False],
       [False],
       [False]])
```

```
from sklearn.metrics import confusion_matrix,accuracy_score
```

```
print('Confusion Matrix')
cm=confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[2332  317]
 [  84  267]]
```

```
print('Accuracy Score')
print(accuracy_score(y_pred,y_test))
```

```
Accuracy Score
0.8663333333333333
```